Informe final - TP Unidad 6

Food Store - Respaldo, monitoreo, mantenimiento y restauracion

- **Integrantes:** Franco Rios y Santino Barone
- **Motor:** PostgreSQL 17.11
- **Base operada:** `foodstore_u6`
- **Fecha de ejecucion:** 2026-11-08
- **Herramientas:** OpenCode, psql, pg_dump, pg_restore, PowerShell 5.1

---

1. Resumen ejecutivo

Se respaldo la base `foodstore_u6` en formato custom, se ejecuto un health check de 8 controles, se hizo una revision de mantenimiento y se restauro el respaldo en la base aislada `foodstore_restore_test`. Todas las validaciones finales pasaron.

Indicador	Resultado
Fecha/hora del respaldo	2026-11-08 15:50:19
Archivo de respaldo	`backups/foodstore_u6_20261108_155019.dump`
Tamano del respaldo	10.292.285 bytes (9,82 MB)
Duracion de `pg_dump`	1,9 s (codigo 0)
Health check inicial	1 FAIL: 46,34 % de tuplas muertas en `pedido`
Health check posterior	8/8 controles OK
Tiempo total restauracion + validacion (RTO)	**5,69 s**
Antiguedad del respaldo al validar (RPO observado)	**0,02 h** (~1 min)
RPO/RTO objetivo	<= 24 h / <= 1 h
Validacion de conteos e integridad	12/12 OK

---

2. Especificacion y automatizacion

La especificacion completa esta en `SPEC_U6.md`. Resumen:

Actividad	Frecuencia	Script	Condicion de terminacion
Respaldo completo	Diaria 02:00	`backup_foodstore.ps1`	Codigo 0 solo si `pg_dump` retorna 0 y el `.dump` existe y no esta vacio
Health check	Diaria 08:00	`health_check.sql`	Codigo 0 si ningun control queda en `FAIL`; el bloque `DO` lanza excepcion si hay fallos
Mantenimiento	Semanal lunes 03:00	`maintenance.sql`	Codigo 0 si `VACUUM (ANALYZE)` y `ANALYZE` terminan y no hay indices invalidos
Prueba de restauracion	Semanal lunes 04:00	`restore_test.ps1`	Codigo 0 solo si `createdb`, `pg_restore --no-owner --no-privileges` y todas las validaciones pasan

La automatizacion se implementa con el Programador de tareas de Windows (o `cron` en Linux) invocando los scripts. Los scripts no contienen contrasenas: usan `PGPASSWORD`, `pgpass.conf` o autenticacion local.

---

3. Respaldo

Comando equivalente:

```
pg_dump -h localhost -p 55436 -U postgres -d foodstore_u6 `
  -Fc --no-password -f backups/foodstore_u6_20261108_155019.dump
```

Resultado real:

- Archivo custom generado: `foodstore_u6_20261108_155019.dump`.
- Tamano: 10.292.285 bytes (9,82 MB).
- `pg_dump` finalizo con codigo **0** en **1,9 s**.
- Log: `logs/backup_20261108_155019.log`.
- Manifiesto de conteos pre-respaldo:

`backups/foodstore_u6_20261108_155019.counts.json`.

- Conteos registrados: cliente 20.000, producto 50.000, pedido 200.000,

detalle_pedido 399.994.

- Verificacion del dump con `pg_restore -l`: codigo **0**

(`docs/evidencia_respaldo.txt`).

- **Prueba negativa:** ejecutar el respaldo contra una base inexistente

devolvio codigo **1** y dejo el error registrado (`docs/evidencia_respaldo_fallo.txt`).

El script valida que el archivo exista y no este vacio; si `pg_dump` falla, borra el archivo parcial, registra el error y devuelve codigo 1.

---

4. Health check

Se ejecutaron 8 controles (`health_check.sql`):

#	Control	Umbral	Resultado inicial	Resultado posterior
1	Tamano de la base	<= 10 GB	94 MB OK	94 MB OK
2	Conexiones totales	<= 80 % de `max_connections`	1 OK	1 OK
3	Consultas activas > 5 min	<= 2	0 OK	0 OK
4	Sesiones `idle in transaction`	<= 5	0 OK	0 OK
5	Peor porcentaje de tuplas muertas	< 20 %	**46,34 % FAIL**	0,00 % OK
6	Indices invalidos	= 0	0 OK	0 OK
7	Locks esperando	= 0	0 OK	0 OK
8	Tablas sin `ANALYZE`	= 0	WARN: tabla temporal del chequeo	0 OK

Evidencias:

- `docs/evidencia_health_check_antes.txt` (detecta el problema).
- `docs/evidencia_health_check.txt` (post-mantenimiento, 8/8 OK).

El control 8 excluye explicitamente la tabla temporal del propio health check para no generar un falso WARN.

---

5. Mantenimiento

Estado observado antes de la accion: la tabla `pedido` tenia 172.684 tuplas muertas sobre 372.684 (46,34 %), producto del `UPDATE` masivo que recalculo los totales durante la carga. El autovacuum no habia actuado aun.

Accion ejecutada:

```
VACUUM (ANALYZE, VERBOSE) cliente;
VACUUM (ANALYZE, VERBOSE) producto;
VACUUM (ANALYZE, VERBOSE) pedido;
VACUUM (ANALYZE, VERBOSE) detalle_pedido;
```

`VACUUM (ANALYZE)` no bloquea lecturas ni escrituras y actualiza estadisticas. En `pedido` se reclamaron 172.684 identificadores muertos en 1.667 paginas de indice (51,66 % del total). El estado posterior mostro **0 tuplas muertas** en las cuatro tablas.

Ajuste preventivo: se configuro autovacuum especifico para la tabla de mayor volumen:

```
ALTER TABLE detalle_pedido SET (
    autovacuum_vacuum_scale_factor = 0.05,
    autovacuum_analyze_scale_factor = 0.02
);
```

Con los valores por defecto (`scale_factor = 0.20`) harian falta ~80.000 tuplas muertas para disparar el vacuum; con 0,05 se dispara antes.

No se uso `VACUUM FULL`: toma `ACCESS EXCLUSIVE`, reescribe la tabla completa y bloquea ventas. Queda reservado a incidentes puntuales de bloat extrema, nunca como tarea periodica.

Evidencia: `docs/evidencia_mantenimiento.txt` (codigo de salida 0).

---

6. Prueba de restauracion

Procedimiento:

1. Se tomo el respaldo mas reciente.
2. Se elimino, si existia, `foodstore_restore_test`.
3. Se creo la base con `createdb`.
4. Se restauro con:

```
pg_restore -h localhost -p 55436 -U postgres -d foodstore_restore_test `
  --no-owner --no-privileges --exit-on-error `
  backups/foodstore_u6_20261108_155019.dump
```

1. Se validaron conteos e integridad contra el manifiesto pre-respaldo.

Tiempos reales:

- Inicio (antes de `createdb`): 2026-11-08 15:51:14.065.
- Fin de validacion: 2026-11-08 15:51:19.759.
- `pg_restore`: 3,01 s.
- **Tiempo total restauracion + validacion (RTO medido): 5,69 s.**
- **Antiguedad del respaldo al validar (RPO observado): 0,02 h.**
- Objetivos: RPO <= 24 h, RTO <= 1 h. Ambos cumplidos.

Validacion de conteos:

Tabla	Esperado	Restaurado	Estado
cliente	20.000	20.000	OK
producto	50.000	50.000	OK
pedido	200.000	200.000	OK
detalle_pedido	399.994	399.994	OK

Validacion de integridad (todos en 0):

- Productos invalidos (`precio < 0` o `stock < 0`): 0.
- Detalles invalidos (`cantidad <= 0` o `precio_unitario <= 0`): 0.
- Pedidos huerfanos de cliente: 0.
- Detalles huerfanos de pedido: 0.
- Detalles huerfanos de producto: 0.
- Totales de pedido inconsistentes con la suma de detalles: 0.
- Indices invalidos: 0.
- Tablas base faltantes: 0.

Evidencia: `docs/evidencia_restauracion.txt` y `logs/restore_20261108_155113.log`.

---

7. Problema real detectado y resolucion

Problema: el health check inicial fallo con **46,34 % de tuplas muertas en `pedido`** (172.684 tuplas) inmediatamente despues de la carga masiva. La causa fue el `UPDATE` que recalculo el total de 172.684 pedidos, que dejo versiones anteriores de las filas pendientes de reclamar. Sin intervencion, ese bloat degrada el rendimiento y desperdicia espacio.

Resolucion: se ejecuto `VACUUM (ANALYZE)` sobre las cuatro tablas y se verifico el estado posterior: 0 tuplas muertas y 8/8 controles OK. Ademas se bajaron los umbrales de autovacuum de `detalle_pedido` para prevenir recurrencia en la tabla de mayor volumen.

Problema secundario de implementacion: la primera corrida de `restore_test.ps1` aborto porque PowerShell 5.1 trataba un `NOTICE` de `dropdb` en `stderr` como error terminante (`$ErrorActionPreference = "Stop"`). Se corrigio usando `Continue` y controlando explicitamente los codigos de salida de cada herramienta. La corrida posterior paso completa.

---

8. Mejoras concretas

1. **Implementada:** autovacuum con umbrales reducidos en `detalle_pedido`

(`0.05` / `0.02`) para que actue antes en la tabla de mayor volumen.

1. **Recomendada:** programar las cuatro tareas y centralizar alertas cuando el

codigo de salida sea distinto de 0 (correo o webhook).

1. **Recomendada:** copiar los `.dump` a almacenamiento externo cifrado y

aplicar retencion 7 diarios + 4 semanales.

1. **Recomendada:** tomar los conteos dentro de la misma instantanea del dump

(`pg_dump --snapshot`) o en una ventana sin escrituras, para eliminar la pequena carrera entre manifiesto y respaldo.

1. **Recomendada:** agregar `pg_verifybackup` / `pg_restore -l` como control

previo obligatorio antes de cada restauracion.

1. **Recomendada:** probar la restauracion en otro servidor o contenedor, no

solo en el mismo cluster, para validar un escenario real de desastre.

---

9. Asistencia de OpenCode y verificacion

OpenCode se uso para:

- Relevar el esquema y los datos disponibles en el repositorio.
- Redactar `SPEC_U6.md`, los scripts y este informe.
- Ejecutar realmente el flujo completo en un cluster temporal de PostgreSQL 17

en el puerto 55436: crear `foodstore_u6`, cargar esquema y datos, respaldar, ejecutar health check, mantenimiento y restaurar.

- Detectar y corregir dos fallas reales durante la ejecucion: la ambiguedad de

`max_dead_pct` en el health check y el tratamiento de `stderr` en PowerShell.

- Verificar resultados leyendo las salidas de `psql`, `pg_dump` y `pg_restore`,

no solo asumiendo que los scripts funcionaban.

Verificacion independiente realizada por el agente:

- `pg_dump` codigo 0 y archivo no vacio.
- `pg_restore -l` codigo 0.
- Health check inicial con 1 FAIL real y posterior con 0 FAIL / 0 WARN.
- `maintenance.sql` codigo 0 y 0 tuplas muertas despues.
- `restore_test.ps1` codigo 0, 4/4 conteos y 8/8 controles de integridad OK.
- RTO y RPO calculados a partir de marcas de tiempo reales.

Las respuestas y decisiones de la IA quedan documentadas en `cambios/DUIA_TP_U6.md`. No se realizaron commits: los archivos quedan para revision y commit del equipo.

La version imprimible de este informe esta en `informe_u6.html`; desde un navegador se puede usar **Imprimir > Guardar como PDF** para obtener el PDF.

---

10. Evidencias

Archivo	Contenido
`backups/foodstore_u6_20261108_155019.dump`	Respaldo custom real
`backups/foodstore_u6_20261108_155019.counts.json`	Conteos pre-respaldo
`logs/backup_20261108_155019.log`	Log del respaldo, codigo 0
`logs/restore_20261108_155113.log`	Log de restauracion y validaciones
`docs/evidencia_respaldo.txt`	Metadata + `pg_restore -l` + log
`docs/evidencia_respaldo_fallo.txt`	Prueba negativa: base inexistente, codigo 1
`docs/evidencia_health_check_antes.txt`	Health check con el FAIL real
`docs/evidencia_health_check.txt`	Health check posterior 8/8 OK
`docs/evidencia_mantenimiento.txt`	Revision de autovacuum y VACUUM
`docs/evidencia_restauracion.txt`	RTO/RPO, conteos e integridad
`docs/verificacion_independiente_restauracion.txt`	Consulta directa a la base restaurada: 4 conteos, 0 invalidos y 9 FK/CHECK